LAPORAN PRATIKUM PEKAN 1

STRUKTUR DATA

disusun Oleh:

ALIF MUHAMMAD IHSAN

NIM 2511531004

Dosen Pengampu : Dr.Wahyudi S.T, M.T

Asisten : M Galid Avero

DEPARTEMEN INFORMATIKA

FAKULTAS TEKNOLOGI INFORMASI

UNIVERSITAS ANDALAS

PADANG

2025

KATA PENGANTAR

Puji syukur ke hadirat Allah SWT atas segala rahmat dan karunia-Nya sehingga penulis dapat menyelesaikan laporan praktikum **Struktur Data** ini dengan baik dan tepat waktu. Laporan ini disusun sebagai salah satu bentuk pertanggungjawaban atas kegiatan praktikum yang telah dilaksanakan, sekaligus sebagai sarana untuk memperdalam pemahaman mengenai konsep dasar struktur data, implementasi algoritma, serta penerapan logika pemrograman dalam kehidupan nyata.

Dalam penyusunan laporan ini, penulis menyadari masih terdapat banyak kekurangan, baik dari segi isi maupun penyajian. Oleh karena itu, penulis sangat mengharapkan kritik dan saran yang membangun demi perbaikan di masa mendatang.

Ucapan terima kasih penulis sampaikan kepada:

- Dosen pengampu mata kuliah **Struktur Data** yang telah memberikan arahan dan bimbingan.
- Asisten praktikum yang senantiasa membantu dalam memahami materi dan menyelesaikan permasalahan teknis.
- Rekan-rekan mahasiswa yang turut berdiskusi dan bekerja sama selama kegiatan praktikum berlangsung.

Akhir kata, semoga laporan ini dapat memberikan manfaat, baik bagi penulis sendiri maupun bagi pembaca yang ingin mempelajari lebih lanjut mengenai struktur data.

Daftar Isi

KATA PENGANTAR i

BAB I 3

PENDAHULUAN 3

1.1 Latar Belakang 3

1.2 Tujuan 3

1.3 Manfaat Praktikum 4

BAB II 5

PEMBAHASAN 5

2.1 Kode Program 3

2.2 Langkah Kerja 11

2.3 Analisis Hasil 19

BAB III 19

PENUTUP 19

3.1 Kesimpulan 19

3.2 Saran 19

DAFTAR PUSTAKA 20

BAB I

PENDAHULUAN

1.1 Latar Belakang

Struktur data merupakan salah satu aspek fundamental dalam ilmu komputer yang berperan penting dalam pengelolaan dan pengorganisasian data. Dengan adanya struktur data, proses penyimpanan, pengambilan, serta manipulasi data dapat dilakukan secara lebih efisien dan terstruktur. Pemahaman mengenai struktur data menjadi dasar bagi pengembangan algoritma yang optimal, sehingga sangat berpengaruh terhadap performa suatu program maupun sistem.

Melalui praktikum struktur data, mahasiswa tidak hanya mempelajari konsep teoritis, tetapi juga mengimplementasikan langsung berbagai bentuk struktur data seperti array, linked list, stack, queue, dan tree dalam bahasa pemrograman. Praktikum ini bertujuan untuk melatih keterampilan analisis logika, pemecahan masalah, serta kemampuan debugging yang diperlukan dalam pengembangan perangkat lunak.

Selain itu, praktikum ini juga memberikan pengalaman nyata dalam menghubungkan teori dengan praktik, sehingga mahasiswa dapat memahami bagaimana suatu algoritma bekerja di balik layar dan bagaimana pemilihan struktur data yang tepat dapat memengaruhi efisiensi program. Dengan demikian, kegiatan praktikum struktur data menjadi sarana penting untuk membangun fondasi keilmuan yang kuat dalam bidang informatika.

1.2 Tujuan

1. Memahami konsep dasar struktur data dan penerapannya dalam pemrograman.
2. Melatih keterampilan dalam mengimplementasikan berbagai jenis struktur data seperti array, linked list, stack, queue, dan tree menggunakan bahasa pemrograman.
3. Mengembangkan kemampuan analisis logika dan pemecahan masalah melalui studi kasus yang diberikan dalam praktikum.
4. Melatih keterampilan debugging serta troubleshooting program agar dapat menemukan dan memperbaiki kesalahan secara sistematis.

1.3 Manfaat Praktikum

1. Memberikan pengalaman langsung dalam mengimplementasikan struktur data sehingga mahasiswa lebih mudah dalam memahami konsep yang abstrak
2. Meningkatkan keterampilan pemograman, khususnya dalam efesiensi algoritma dan pengelolaan data.
3. Menjadi bekal penting untuk mata kuliah lanjutan maupun proyek pengembangan perangkat lunak di masa depan.
4. Membantu mahasiswa memahami pentingnya pemilihan struktur data yang tepat dalam meningkatkan peforma program.

BAB II

PEMBAHASAN

2.1 Kode Progaram

a. Kode Program jam_2511531004

```
public class jam_2511531004 {

  private int hh; // 0...23
  private int mm; // 0...59
  private int ss; // 0...59

  // validator
  public static boolean isValid(int h , int m ,int s ) {
        return ( 0 <= h && h <= 23) && (0 <= m && m <= 59) && (0 <= s && s <= 59) ;
  }

  // konstruktor
  public jam_2511531004 (int h , int m , int s ) {
        this.hh = h;
        this.mm = m;
        this.ss = s;

  }

  // selektor (getter)
  public int getHour () {
        return hh;
  }
  public int getMinute() {
        return mm;
  }
  public int getSecond() {
        return ss;
  }

  // mutator (setter)
  public void setHour (int h) {
        this.hh = h;
  }
  public void setMinute(int m) {
        this.mm = m;
  }
  public void setSecond(int s ) {
        this.ss = s;
  }
```

```java

    // konversi
    public int toSeconds () { return hh*3600 + mm*60 + ss;}
    public static jam_2511531004 fromSeconds(int total ) {
            if ( total < 0 ) throw new IllegalArgumentException("detikn negatif") ;
            total %= 24*3600; // wrap 24 jam
            int h = total/3600; total %= 3600;
            int m = total/60; int s = total % 60;
            return new jam_2511531004(h,m,s);

    }
    // relasional
    public int compareTo( jam_2511531004 other ) {return Integer.compare(this.toSeconds(), other.toSeconds() ); }
    public boolean equals(Object o) {
            if ( ! (o instanceof jam_2511531004 j )) return false;
            return hh == j.hh && mm == j.mm && ss == j.ss;

    }
    public int hashCode () {
            return java.util.Objects.hash(hh, mm, ss );
    }


    // aritmatika
    public jam_2511531004 plus (jam_2511531004 other) {
            return fromSeconds(this.toSeconds () + other.toSeconds() );
    }
    public jam_2511531004 minus (jam_2511531004 other ) {
            return fromSeconds(Math.floorMod(this.toSeconds () - other.toSeconds(), 24*3600));
    }
    public jam_2511531004 nextSecond () {
            return fromSeconds ( this.toSeconds () + 1);
    }
    public jam_2511531004 nextNSecond (int n ) {
            return fromSeconds (this.toSeconds () + Math.max(0,n) );
    }
    public jam_2511531004 prevSecond () {
            return fromSeconds (Math.floorMod ( this.toSeconds () - 1, 24*3600));
    }
    public jam_2511531004 prevNSecond (int n) {
            return fromSeconds (Math.floorMod(this.toSeconds() - Math.max(0,n) , 24*3600)) ;
    }

    // durasi  (detik --> bisa negatif jika this --> other? sesuai spesifikasi duari
    public static int durasiDetik(jam_2511531004 jaw, jam_2511531004 jakh) {
            return jakh.toSeconds() - jaw.toSeconds() ; }
            // overriding ke String
    public String toString () {
            return String.format("%02d: %02d: %02d " , hh, mm ,ss  ) ;
    }
```

```
    }

```

b. Kode Program jamDriver_2511531004

```
package pekan1_2511531004;

public class jamDriver_2511531004 {
        public static void main(String[] args) {
                jam_2511531004 a = new jam_2511531004(23, 59, 50) ;
                jam_2511531004 b = new jam_2511531004(0,0,15);
                System.out.println("a =  " + a);
                System.out.println("b = " + b );
                System.out.println("a + b = " + a.plus(b));
                System.out.println("next 20s = " + a.nextNSecond
(20));
                System.out.println("durasi (a,b) " +
jam_2511531004.durasiDetik(a,b));
                System.out.println("a.compareTo(b) =  " +
a.compareTo(b));



        }

}
```

c. Kode Program jamDriver2_2511531004

```
package pekan1_2511531004;
import java.util.Scanner;
public class jamDriver2_2511531004 {

        public static void main (String [] args ) {

                Scanner input = new Scanner(System.in) ;
                System.out.println("==== Program Driver Objek jam ====");

                // 1. input jam pertama
                System.out.println("\n ---- input jam 1 ----");
                jam_2511531004 j1 = buatJamDariInput(input);

                // 2. input jam kedua
                System.out.println("\n ---- input jam 2 ----");
                jam_2511531004 j2 = buatJamDariInput(input);

                // 3. menampilkan data
                System.out.println("\n ---- hasil operasi ----");
                System.out.println("jam 1 (String) : "  + j1.toString());
                System.out.println("jam 2 (String) : " + j2.toString());
                System.out.println("jam 1 dalam detik : " +
j1.toSeconds());
```

```
            System.out.println("Jam 2 dalam detik : " +
j2.toSeconds());


            // 4. menampilkan relasional (perbandingan)
            int perbandingan = j1.compareTo(j2) ;
            if (perbandingan > 0 ) {
                  System.out.println("Status :  jam 1 lebih lambat
(setelah) jam 2 ");
            } else if (perbandingan < 0) {
                  System.out.println("Status : jam 1 lebih awal
(sebelum) jam 2");
            } else {
                  System.out.println("Status : jam 1 dan jam 2 sama
sama persisi");


            }


            // 5. operasi aritmatika
            System.out.println("Durasi (j1 ke j2) : " +
jam_2511531004.durasiDetik(j1, j2) + "detik");
            jam_2511531004 jNext = j1.nextSecond();
            System.out.println("jam 1 detik berikutnya : " + jNext);

            jam_2511531004 jPrev = j1.prevSecond();
            System.out.println("Jam 1 detik sebelumnya : " + jPrev);


            // 6. operasi penjumlahan jam
            jam_2511531004 jHasilPlus = j1.plus(j2);
            System.out.println("Hasil J1 + J2 : " + jHasilPlus);


            input.close();
      }

      public static jam_2511531004 buatJamDariInput(Scanner input ) {
            int h, m, s ;
            while ( true ) {
                  System.out.println("Masukkan jam ( 0-23) : " );
                  h = input.nextInt();
                  System.out.println("Masukkan Menit (0-59) : ");
                  m = input.nextInt();
                  System.out.println("Masukkan Detik (0-59) : " );
                  s = input.nextInt();



                  // memanggil method static isValid dari kelas
jam_2511531004
                  if (jam_2511531004.isValid(h, m, s)) {
                        return new jam_2511531004 (h,m,s);

                  } else {
```

```
                        System.out.println("[error] input tidak valid
silahkan ulangi. \n");
                        }
                    }
            }

        }
```

2.2 Langkah Kerja

a) Langkah kerja jam_2511531004

1. **Deklarasi Atribut**
   - Program mendefinisikan tiga variabel privat:
     - hh → jam (0–23)
     - mm → menit (0–59)
     - ss → detik (0–59)
2. **Validasi Input**
   - Method isValid(h, m, s) mengecek apakah nilai jam, menit, dan detik berada dalam rentang yang benar.
   - Mengembalikan true jika valid, false jika tidak.
3. **Konstruktor**
   - jam_2511531004(int h, int m, int s) digunakan untuk membuat objek jam baru dengan nilai jam, menit, dan detik tertentu.
4. **Selektor (Getter)**
   - getHour(), getMinute(), getSecond() → mengambil nilai jam, menit, dan detik dari objek.
5. **Mutator (Setter)**
   - setHour(h), setMinute(m), setSecond(s) → mengubah nilai jam, menit, dan detik pada objek.
6. **Konversi**
   - toSeconds() → mengubah jam, menit, detik menjadi total detik.
   - fromSeconds(total) → mengubah total detik menjadi format jam:menit:detik dengan pembatasan 24 jam.
7. **Relasional**

- compareTo(other) → membandingkan dua objek jam berdasarkan total detik.
- equals(Object o) → mengecek apakah dua objek jam sama persis.
- hashCode() → menghasilkan kode hash berdasarkan jam, menit, detik.

8. **Operasi Aritmatika**
    - plus(other) → menjumlahkan dua objek jam (dalam detik).
    - minus(other) → mengurangi dua objek jam dengan hasil dalam format jam.
    - nextSecond() → menambah 1 detik.
    - nextNSecond(n) → menambah n detik.
    - prevSecond() → mundur 1 detik.
    - prevNSecond(n) → mundur n detik.
9. **Durasi**
    - durasiDetik(jaw, jakh) → menghitung selisih detik antara dua objek jam (jaw → jakh).
10. **Output String**
    - toString() → menampilkan jam dalam format HH:MM:SS.

b) jamDriver_2511531004

1. **Inisialisasi Objek Jam**
    - a = new jam_2511531004(23, 59, 50) → membuat objek jam dengan nilai 23:59:50.
    - b = new jam_2511531004(0, 0, 15) → membuat objek jam dengan nilai 00:00:15.
2. **Menampilkan Objek Jam**
    - System.out.println("a = " + a) → memanggil toString() dari objek a, hasilnya "23:59:50".
    - System.out.println("b = " + b) → memanggil toString() dari objek b, hasilnya "00:00:15".
3. **Operasi Penjumlahan Jam**
    - a.plus(b) → menjumlahkan total detik dari a dan b, lalu dikonversi kembali ke format jam.

        - a = 23:59:50 → 86390 detik
        - b = 00:00:15 → 15 detik
        - Total = 86405 detik → dibungkus ke 24 jam → hasil 00:00:05.

4. **Operasi Penambahan Detik**
    - a.nextNSecond(20) → menambah 20 detik ke jam a.
        - a = 23:59:50 → +20 detik = 24:00:10 → wrap ke 00:00:10.
5. **Menghitung Durasi**
    - jam_2511531004.durasiDetik(a, b) → menghitung selisih detik antara a dan b.
        - b = 15 detik, a = 86390 detik.
        - Hasil = 15 − 86390 = **−86375 detik**.
6. **Membandingkan Objek Jam**
    - a.compareTo(b) → membandingkan total detik a dan b.
        - Karena a (86390 detik) lebih besar dari b (15 detik), hasilnya **positif (1)**.

c) jamDriver2_2511531004

1. **Inisialisasi Scanner**
    - Program membuka Scanner input = new Scanner(System.in) untuk membaca data dari keyboard.
2. **Menampilkan Judul Program**
    - System.out.println("==== Program Driver Objek jam ===="); → memberi informasi awal ke pengguna.
3. **Input Jam Pertama**
    - Program memanggil method buatJamDariInput(input) untuk meminta pengguna memasukkan jam, menit, dan detik.
    - Method ini akan:
        - Meminta input jam (0–23), menit (0–59), dan detik (0–59).
        - Mengecek validitas dengan jam_2511531004.isValid(h, m, s).
        - Jika valid → membuat objek jam_2511531004.

        - Jika tidak valid → menampilkan pesan error dan mengulang input.

4. **Input Jam Kedua**
    - Proses yang sama dilakukan untuk jam kedua (j2).
5. **Menampilkan Data Jam**
    - Program menampilkan:
        - Jam 1 dan Jam 2 dalam format string (toString()).
        - Jam 1 dan Jam 2 dalam bentuk total detik (toSeconds()).
6. **Relasional (Perbandingan)**
    - Program membandingkan j1.compareTo(j2):
        - Jika hasil > 0 → jam 1 lebih lambat (setelah jam 2).
        - Jika hasil < 0 → jam 1 lebih awal (sebelum jam 2).
        - Jika hasil = 0 → jam 1 dan jam 2 sama persis.
7. **Operasi Aritmatika**
    - durasiDetik(j1, j2) → menghitung selisih detik antara jam 1 dan jam 2.
    - j1.nextSecond() → menampilkan jam 1 detik berikutnya.
    - j1.prevSecond() → menampilkan jam 1 detik sebelumnya.
8. **Operasi Penjumlahan Jam**
    - j1.plus(j2) → menjumlahkan jam 1 dan jam 2 dalam bentuk detik, lalu dikonversi kembali ke format jam.
9. **Menutup Scanner**
    - input.close(); → menutup scanner agar tidak terjadi memory leak.

BAB III

PENUTUP

3.1 Kesimpulan

d) Praktikum ini berhasil menunjukkan bagaimana konsep **kelas dan objek** dalam Java dapat digunakan untuk merepresentasikan waktu (jam, menit, detik) secara terstruktur.
e) Implementasi method seperti **getter, setter, konversi, relasional, dan aritmatika** membuktikan bahwa operasi terhadap data dapat dilakukan dengan cara yang sistematis dan efisien.
f) Program driver (jamDriver dan jamDriver2) memperlihatkan bagaimana objek jam dapat diuji melalui input langsung maupun simulasi, sehingga mahasiswa dapat memahami hubungan antara teori struktur data dengan implementasi nyata.
g) Dengan adanya validasi input, program mampu mencegah kesalahan data sehingga hasil operasi lebih akurat dan sesuai spesifikasi.
h) Praktikum ini melatih keterampilan analisis logika, debugging, serta pemahaman konsep **encapsulation** dan **object-oriented programming (OOP)**.

3.2 Saran

1. Sebaiknya mahasiswa lebih banyak melakukan **eksperimen** dengan me nambahkan fitur tambahan, misalnya format 12 jam (AM/PM) atau perhitungan selisih hari, agar pemahaman semakin mendalam.
2. Perlu dilakukan **uji coba dengan berbagai kasus input**, termasuk input ekstrem (misalnya jam = 23:59:59) untuk memastikan program benar-benar robust.
3. Dokumentasi kode sebaiknya dibuat lebih detail dengan komentar yang konsisten agar mudah dipahami oleh pembaca lain.
4. Praktikum berikutnya dapat dikembangkan dengan mengintegrasikan kelas jam ini ke dalam aplikasi yang lebih besar, misalnya sistem penjadwalan atau kalender sederhana.

5. Mahasiswa diharapkan terus melatih kemampuan debugging dan analisis error karena hal ini sangat penting dalam pengembangan perangkat lunak di dunia nyata.

DAFTAR PUSTAKA

1. Deitel, H. M., & Deitel, P. J. (2017). *Java: How to Program (Early Objects)*. Pearson Education.
2. Goodrich, M. T., Tamassia, R., & Goldwasser, M. H. (2014). *Data Structures and Algorithms in Java*. Wiley.
3. Malik, D. S. (2012). *Data Structures Using Java*. Cengage Learning.
4. Oracle. (2026). *Java Documentation*. Diakses dari: https://docs.oracle.com/javase
5. Modul Praktikum Struktur Data, Program Studi Informatika, Universitas Andalas (2026).

Tugas Praktikum Struktur Data Pekan 1

1. Kode Program

   a. Mobil_2511531004

```
package Struktur_data;

public class mobil_2511531004 {

        private String nama;
        private int tahun;
        private int cc;
        private int harga;
        private String merk;


        // konstruktor
        mobil_2511531004( String nama, int tahun, int cc, int harga, String merk) {
                this.nama = nama;
                this.tahun = tahun;
                this.cc = cc;
                this.harga = harga;
                this.merk = merk;
        }

        // getter
        String getNama() {
                return nama;
        }
        int getTahun() {
                return tahun;
        }
        int getCc() {
                return cc;
        }
        int getHarga() {
                return harga;
        }
        String getMerk() {
                return merk;
        }

        // setter
        public void setNama(String nama) {
                this.nama = nama;
        }
        public void setTahun(int tahun) {
                this.tahun = tahun;
        }
        public void setCc(int cc) {
                this.cc = cc;
        }
        public void setHarga(int harga) {
                this.harga = harga;
```

```
        }
        public void setMerk(String merk) {
                this.merk = merk;
        }

        public void tampilkanInfo() {
                System.out.println(nama + " " + tahun + " " + cc + " " + harga + " " + merk);
        }


}
```

b. driver_2511531004

```
package Struktur_data;

public class driver_2511531004 {

        static mobil_2511531004[] daftarMobil = new mobil_2511531004[10];
        static int jumlahMobil = 0;

        public static void tambahMobil(String nama, int tahun, int cc, int harga, String merk) {

                daftarMobil[jumlahMobil] = new mobil_2511531004(nama,tahun,cc,harga,merk) ;
                jumlahMobil++;

        }

        static void hapusMobil(int index) {
                for ( int i = index; i < jumlahMobil-1; i++) {
                        daftarMobil[i] = daftarMobil[i+1] ;
                }
                jumlahMobil--;
        }
        static void tampilMobil() {
                for(int i=0; i<jumlahMobil; i++) {
                        daftarMobil[i].tampilkanInfo();
                }



                }
        public static void main (String[] args) {
                tambahMobil("Fortuner",2024,2800,550000000,"Toyota");
                tambahMobil("Brio", 2020, 1000, 12000000, "Honda");

                tampilMobil();
        }

}
```

2. Penjelasan Kode
    a. Kode mobil_2511531004

1. **DefinisiKelas**

   Program membuat sebuah kelas bernama mobil_2511531004 yang berfungsi sebagai cetakan (blueprint) untuk objek mobil. Kelas ini berisi data dan perilaku yang menggambarkan sebuah mobil.

2. **AtributMobil**

   Di dalam kelas terdapat beberapa atribut penting: nama mobil, tahun produksi, kapasitas mesin (cc), harga mobil, dan merek mobil. Semua atribut ini disimpan secara privat agar tidak bisa diakses langsung dari luar kelas, sesuai prinsip *encapsulation*.

3. **Konstruktor**

   Konstruktor digunakan untuk membuat objek mobil baru sekaligus mengisi nilai awal dari atribut-atribut tersebut. Jadi saat objek dibuat, langsung terisi dengan data mobil yang lengkap.

4. **Getter(Selektor)**

   Getter adalah metode untuk mengambil nilai dari atribut. Misalnya, jika ingin tahu nama mobil, bisa memanggil getNama(). Getter memastikan data bisa dibaca tanpa harus mengakses langsung variabel privat.

5. **Setter(Mutator)**

   Setter adalah metode untuk mengubah nilai atribut. Misalnya, jika harga mobil berubah, bisa menggunakan setHarga(). Dengan setter, perubahan data tetap terkontrol.

6. **MethodtampilkanInfo**

   Program menyediakan satu metode tambahan untuk menampilkan seluruh informasi mobil dalam satu baris ke layar. Ini memudahkan pengguna melihat data mobil secara ringkas.

b. Kode Program driver_2511531004

1. **DeklarasiArrayMobil**

   Program menyiapkan sebuah array daftarMobil dengan kapasitas 10 elemen. Array ini digunakan untuk menyimpan objek-objek mobil. Ada juga variabel jumlahMobil untuk menghitung berapa mobil yang sudah dimasukkan.

2. **MethodtambahMobil**

   Method ini digunakan untuk menambahkan mobil baru ke dalam array. Saat dipanggil, ia akan membuat objek mobil dengan data yang diberikan (nama, tahun, cc, harga, merk), lalu menyimpannya ke dalam array di posisi jumlahMobil. Setelah itu, nilai jumlahMobil bertambah satu.

3. **MethodhapusMobil**

   Method ini digunakan untuk menghapus mobil dari array berdasarkan indeks. Caranya adalah dengan menggeser semua elemen setelah indeks tersebut ke kiri, sehingga posisi kosong terisi. Setelah itu, jumlahMobil dikurangi satu.

4. **MethodtampilMobil**

   Method ini digunakan untuk menampilkan semua mobil yang ada di dalam array. Program akan melakukan perulangan dari indeks 0 sampai jumlahMobil - 1, lalu memanggil tampilkanInfo() dari setiap objek mobil.

5. **Method main (Program Utama)**

   - Pertama, program menambahkan dua mobil ke dalam array:
     - Mobil pertama: Fortuner, tahun 2024, cc 2800, harga 550 juta, merk Toyota.
     - Mobil kedua: Brio, tahun 2020, cc 1000, harga 12 juta, merk Honda.
   - Setelah itu, program memanggil tampilMobil() untuk menampilkan informasi kedua mobil tersebut ke layar.